Autor: Javier Crespo

Practica 2

ADMINISTRACIÓN TECNOLOGÍAS WEB

Fecha: Noviembre 2024

Índice

Creación de Certificados SSL con Java y Tomcat. (Parte I) 3
2.2. Generación de un certificado autofirmado 3
Preparación del almacén de certificados 4
Edición del fichero de configuración del servidor 5
2.3. Solicitud de certificado a una autoridad certificadora 6
Generación del requerimiento de certificado 6
Envió de solicitud a una entidad certificadora 7
Creación de Certificados SSL con Java y Tomcat. (Parte II) 8
2.4. Generación de Certificados con OpenSSL 8
Verificación de la instalación del certificado 10
Problema encontrado 12

Tabla de ilustraciones

Ilustración 1 3
Ilustración 2 3
Ilustración 3 4
Ilustración 4 5
Ilustración 5 5
Ilustración 6 5
Ilustración 7 7
Ilustración 8 7
Ilustración 9 8
Ilustración 10 9
Ilustración 11 9
Ilustración 12 10
Ilustración 13 11
Ilustración 14 11
Ilustración 15 12

Creación de Certificados SSL con Java y Tomcat. (Parte I)

2.2. Generación de un certificado autofirmado

En esta práctica, el objetivo es generar y configurar un certificado autofirmado para habilitar el acceso seguro mediante HTTPS a un servidor local en Tomcat. Para lograr que el navegador mostrara el mensaje de advertencia de "Conexión no segura", fue necesario editar el archivo de configuración **Server.xml** en el servidor Tomcat. En particular, agregué un conector HTTPS en el puerto 8443, como se muestra en la imagen. Esta configuración permite al servidor aceptar conexiones seguras y provoca la advertencia esperada en el navegador al acceder a https://localhost:8443 , ya que el certificado es autofirmado y no está validado por una autoridad certificadora.

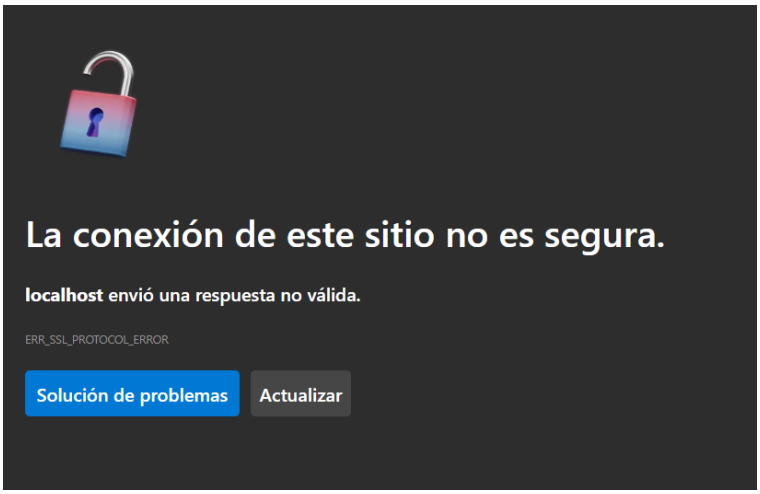

Ilustración 1

```
-->
<Connector port="8080" protocol="HTTP/1.1"
           connectionTimeout="20000"
           redirectPort="8443"
           maxParameterCount="1000"
           />
 <Connector port="8443" protocol="HTTP/1.1"
           connectionTimeout="20000"
           redirectPort="8443"
           maxParameterCount="1000"
           />
```

Ilustración 2

Preparación del almacén de certificados

Para comenzar, crearemos un archivo de almacén de certificados utilizando la herramienta keytool, que viene incluida con la instalación de Java. Este almacén almacenará nuestro certificado autofirmado.

1. **Acceso al directorio de Java**: Primero, accedemos al directorio \bin de Java desde la consola de comandos para ejecutar la herramienta keytool.

2. **Creación del almacén de certificados**: Antes de ejecutar el comando, creamos una carpeta en el sistema llamada C:\atw para guardar nuestro almacén de certificados. Luego, ejecutamos el siguiente comando para generar el certificado:

.\keytool.exe -genkey -alias tomcat -keyalg RSA -keysize 2048 -keystore "C:\atw\.keystorejfuster"

3. **Ingreso de información**: Una vez ejecutado el comando, keytool nos solicitará algunos datos para completar la generación del certificado. Primero, pedirá una contraseña para proteger el almacén de claves; por defecto, usamos la contraseña "changeit".

4. **Información personal y organizativa**: Después, keytool solicitará la siguiente información:

   - **Nombre y apellido**: Nombre completo de la persona o entidad.
   - **Unidad organizativa**: Departamento o unidad dentro de la organización (por ejemplo, "IT").
   - **Organización**: Nombre de la organización o empresa.
   - **Ciudad o localidad**: Ciudad en la que se encuentra la organización.
   - **Estado o provincia**: Estado o provincia.
   - **Código de país de dos letras**: Código ISO del país (por ejemplo, "ES" para España).

5. **Confirmación**: Una vez ingresados estos datos, keytool mostrará un resumen de la información ingresada y nos pedirá confirmación para asegurarse de que todo sea correcto. Si confirmamos, el almacén de certificados se crea en la ruta especificada (C:\atw\.keystorejfuster), listo para su uso en la configuración de nuestro servidor.

```
Use "keytool -?, -h, or --help" for this help message
PS C:\Program Files\Java\jdk-23\bin> .\keytool.exe -genkey -alias tomcat -keyalg RSA -keysize 2048 -keystore "C:\atw\.ke
ysotrejcrespo"
Enter keystore password:
Re-enter new password:
Enter the distinguished name. Provide a single dot (.) to leave a sub-component empty or press ENTER to use the default
value in braces.
What is your first and last name?
What is the name of your organizational unit?
What is the name of your organization?
What is the name of your City or Locality?
What is the name of your State or Province?
What is the two-letter country code for this unit?
Is CN=ATWJavierCrespo.com, OU=AffiliationATW, O=ATW, L=Elche, ST=Alicante, C=ES correct?
  [no]:  yes
Generating 2.048 bit RSA key pair and self-signed certificate (SHA384withRSA) with a validity of 90 days
        for: CN=ATWJavierCrespo.com, OU=AffiliationATW, O=ATW, L=Elche, ST=Alicante, C=ES
PS C:\Program Files\Java\jdk-23\bin>
```

Ilustración 3

Edición del fichero de configuración del servidor

Abrimos de nuevo el archivo server.xml y localizamos la sección correspondiente a la configuración de HTTPS. En esta sección, descomentamos (es decir, quitamos los símbolos <!-- y --> de) el código que se muestra en la figura #, que configura el conector SSL.

Luego, actualizamos la ruta del archivo *keystore* para que apunte al archivo que generamos en el paso anterior. Cambiamos esta línea:
<Certificate certificateKeystoreFile="conf/localhost-rsa.jks"

por la siguiente:
<Certificate certificateKeystoreFile="C:/atw/.keystorejcrespo"

```
<Connector port="8443" protocol="org.apache.coyote.http11.Http11NioProtocol"
           maxThreads="150" SSLEnabled="true"
           maxParameterCount="1000"
           >
    <UpgradeProtocol className="org.apache.coyote.http2.Http2Protocol" />
    <SSLHostConfig>
        <Certificate certificateKeystoreFile="C:/atw/.keysotrejcrespo"
                     certificateKeystorePassword="changeit" type="RSA" />
    </SSLHostConfig>
</Connector>
```

Ilustración 6

Ahora aparece un nuevo mensaje de advertencia en el navegador. Al seleccionar "Continuar en localhost (no seguro)," se muestra la pantalla de bienvenida del servidor Tomcat.

Ilustración 4

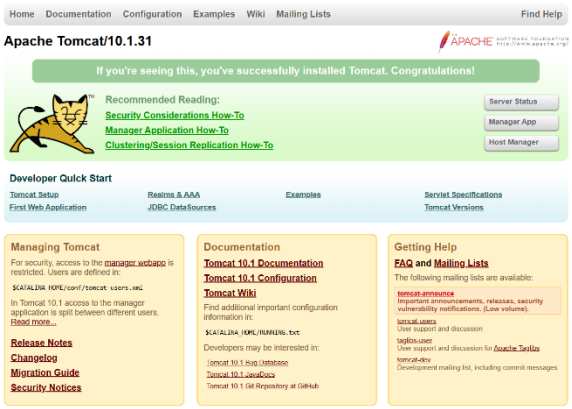

Ilustración 5

2.3. Solicitud de certificado a una autoridad certificadora

Generación del requerimiento de certificado

Para generar el archivo de solicitud de certificado (certreq.csr), utilizamos el siguiente comando en la terminal:

keytool -certreq -keyalg RSA -alias tomcat -file certreq.csr -keystore archivo_keystore

Este comando crea una solicitud de certificado basada en el almacén de claves especificado (indicado en archivo_keystore). Sin embargo, experimenté varios problemas al ejecutar este comando, ya que la consola arrojaba un error indicando que no encontraba el archivo *keystore*. Aunque no recuerdo exactamente la solución final, es importante asegurarse de que el nombre y la ubicación del archivo *keystore* sean correctos.

Una vez solucionado este problema, el comando generó con éxito el archivo certreq.csr en la carpeta C:\atw creada anteriormente. Este archivo contiene la información de la clave pública, que será enviada a una autoridad certificadora para la creación de un certificado SSL.

El contenido de certreq.csr generado es el siguiente:

-----BEGIN NEW CERTIFICATE REQUEST-----
MIIC6jCCAdICAQAwdTELMAkGA1UEBhMCRVMxETAPBgNVBAgTCEFsaWNhbnRlMQ4w
DAYDVQQHEwVFbGNoZTEMMAoGA1UEChMDQVRXMRcwFQYDVQQLEw5BZmZpbGlhdGlv
bkFUVzEcMBoGA1UEAxMTQVRXSmF2aWVyQ3Jlc3BvLmNvbTCCASIwDQYJKoZIhvcN
AQEBBQADggEPADCCAQoCggEBAMmu9QnXVpb/0LpOrkGIjlayttg4vmrU6hc/f0MK
LBd/ZhVOniqqWCTzN9ovJsBsBjlRr26YWlq2NgTZGEwd25c8YYZAeab87k33Ecox
mjQo9vHI597NROB9v+drqFWmRjqqe3criFvL0HEiwKeG1ATF0SY4iFKsXntUGzBl
3XWNMN5MLOymbP+4sjmtAYxlQrdYhbsIfvOuB+a4Rmp+A8Q90umaBG+N9ldeg5K7
Hfg9L2L058hq1kHJq1NktHofVevV/nT0tJjIk98Ihk+JGNEnG1HbzikPxqru4mIb
A0OAT4FFRfjoAO1FEGfgDF2uAytOclnJGl07cBaD+/yGCYMCAwEAAaAwMC4GCSqG
SIb3DQEJDjEhMB8wHQYDVR0OBBYEFBIt71PPZt4nceZaaFOmPuIF2krEMA0GCSqG
SIb3DQEBDAUAA4IBAQCaUVCnBZUBeT0/es+BQtu7fyUsCIOzYJlvNnSdOqf8OxEl
KXo1uvhjLtNWmJ0YwH0EFUv76k2axj3rvdSWY4HOHisY+tor22GzjcVllKkaoVk9
vci+g8mBVXvi2Xpx09IdrPPs7eY+5sLIEa6qbtwD9AtvKTcxR0eBEuZon6DqC8Sn
RXIi1raGOiiaETLTeXSsSBKQlULh2mgkTYWbAZ3brPBeYm7pYyWwmJlZY22JlN5W
jvb32fjVrC4Mj72icT6qAleJtto1VWO9V9Q7LzTWzfu34YTGRD8oXy+/X+6mhUpm
fpIdYW7OgTkZmPikNgxVlc8nkKsaL+DCYhGCrAVH

-----END NEW CERTIFICATE REQUEST-----

Envió de solicitud a una entidad certificadora

Para obtener el certificado SSL, intentamos utilizar la entidad certificadora **SSL247**. Al iniciar el proceso, la entidad primero solicita el nombre del dominio asociado al certificado que estamos generando, en este caso, ATWJavierCrespo.com, que es el mismo dominio que usamos al crear el archivo de almacén de claves (.keystore).

El problema surge cuando SSL247 solicita un correo electrónico de contacto que debe tener el mismo dominio, es decir, un correo que termine en @atwjaviercrespo.com. Dado que este dominio no existe en realidad, no podemos proporcionar un correo válido con esa terminación, lo que nos impide completar el proceso de certificación a través de esta entidad.

En la **Parte 2** de la práctica, abordaremos un método alternativo para generar el certificado de manera manual y sin depender de una entidad certificadora externa.

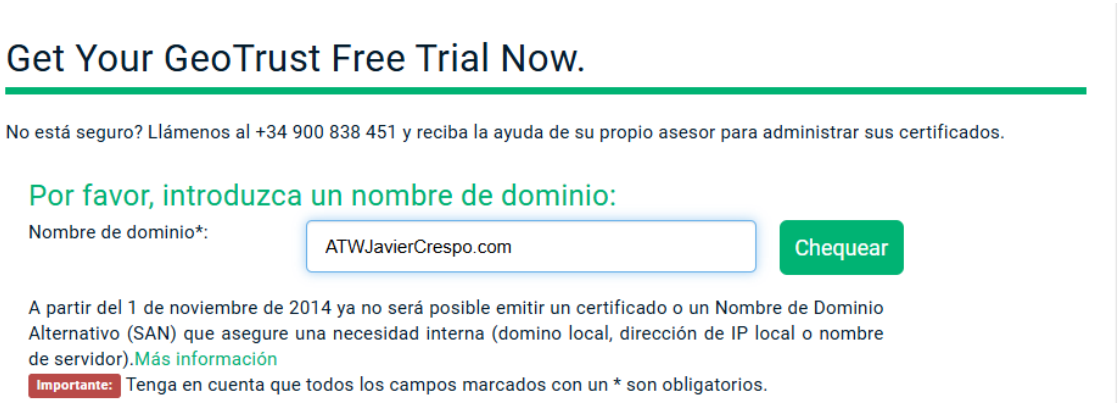

Ilustración 7

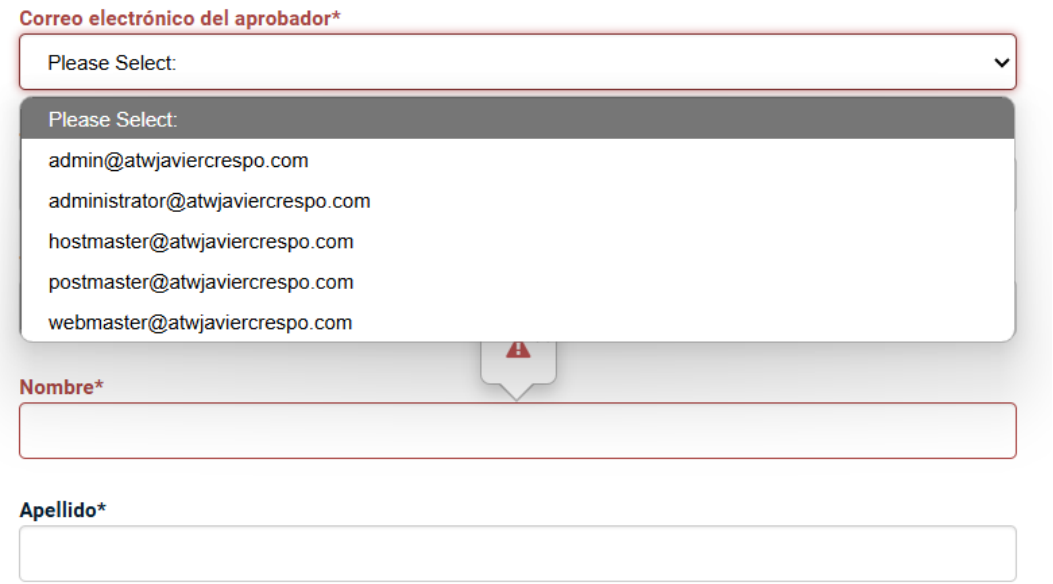

Ilustración 8

Creación de Certificados SSL con Java y Tomcat. (Parte II)

2.4. Generación de Certificados con OpenSSL

Para este apartado, usamos el siguiente comando en la terminal para exportar la clave privada de Tomcat:
java -jar ExportPrivateKey.zip "C:\atw \keysotrejcrespo" JKS "changeit" tomcat ca.key

Este comando utiliza el archivo ExportPrivateKey.zip (disponible en la sección de recursos de esta práctica) para extraer la clave privada desde el almacén de claves (keystorejcrespo) y guardarla en el archivo ca.key. Sin embargo, debido a problemas de compatibilidad con Java, este comando puede no ejecutarse correctamente.

Como alternativa, utilizamos la aplicación **KeyStore Explorer** para extraer la clave privada. Con esta herramienta:

1. Seleccionamos nuestro archivo de almacén de claves (keystorejcrespo).
2. Seguimos los pasos indicados en la práctica, lo que nos permite exportar la clave privada de Tomcat de manera efectiva y guardarla en el archivo ca.key.

KeyStore Explorer simplifica el proceso de exportación y evita los problemas que pueden surgir con el uso directo de Java.

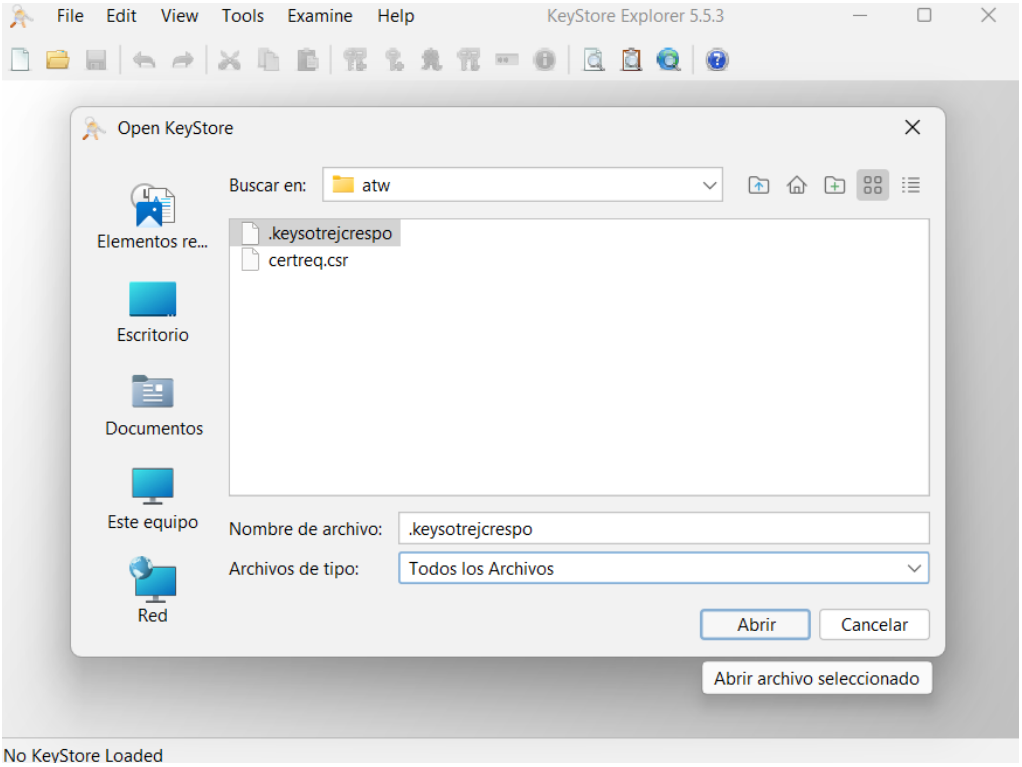

Ilustración 9

En segundo lugar, generamos un certificado con el archivo .csr, que hemos exportado en la parte 1 de esta práctica, y la clave privada (ca.key), para ello debemos utilizar el programa OpenSSL, incluido en los ficheros de esta práctica, y teclear la siguiente orden:

openssl x509 -req -days 365 -in certreq.csr -signkey ca.key -out certificadotomcat.crt

```
C:\Users\Usuario>cd "C:\atw"

C:\atw>openssl x509 -req -days 365 -in certreq.csr -signkey ca.key -out certificadotomcat.crt
Certificate request self-signature ok
subject=C=ES, ST=Alicante, L=Elche, O=ATW, OU=AffiliationATW, CN=ATWJavierCrespo.com

C:\atw>
```

Ilustración 10

Una vez que hemos generado nuestro certificado, procedemos a su instalación. Para ello, seguimos los pasos detallados en la práctica, que explican el proceso de forma clara y detallada. Este proceso garantiza que el certificado se instale correctamente en el servidor y esté listo para habilitar las conexiones seguras.

Ilustración 11

Verificación de la instalación del certificado

Podemos comprobar si la instalación del certificado se realizó correctamente siguiendo estos pasos:

1. Abrir la herramienta de administración de certificados:

- Presionamos Win + R para abrir el cuadro de diálogo "Ejecutar".
- Escribimos certmgr.msc. Esto abrirá la consola de administración de certificados de Windows, que permite ver y administrar los certificados instalados en el sistema.

2. Buscar el certificado instalado:

- En la consola de certmgr.msc, navegamos hasta **Certificados - Usuario actual > Personal > Entidades de certificado raiz**. Aquí es donde los certificados instalados suelen aparecer.
- Buscamos el certificado que genere, que debería mostrarse con el nombre del dominio o alias que use (ATWJavierCrespo.com).

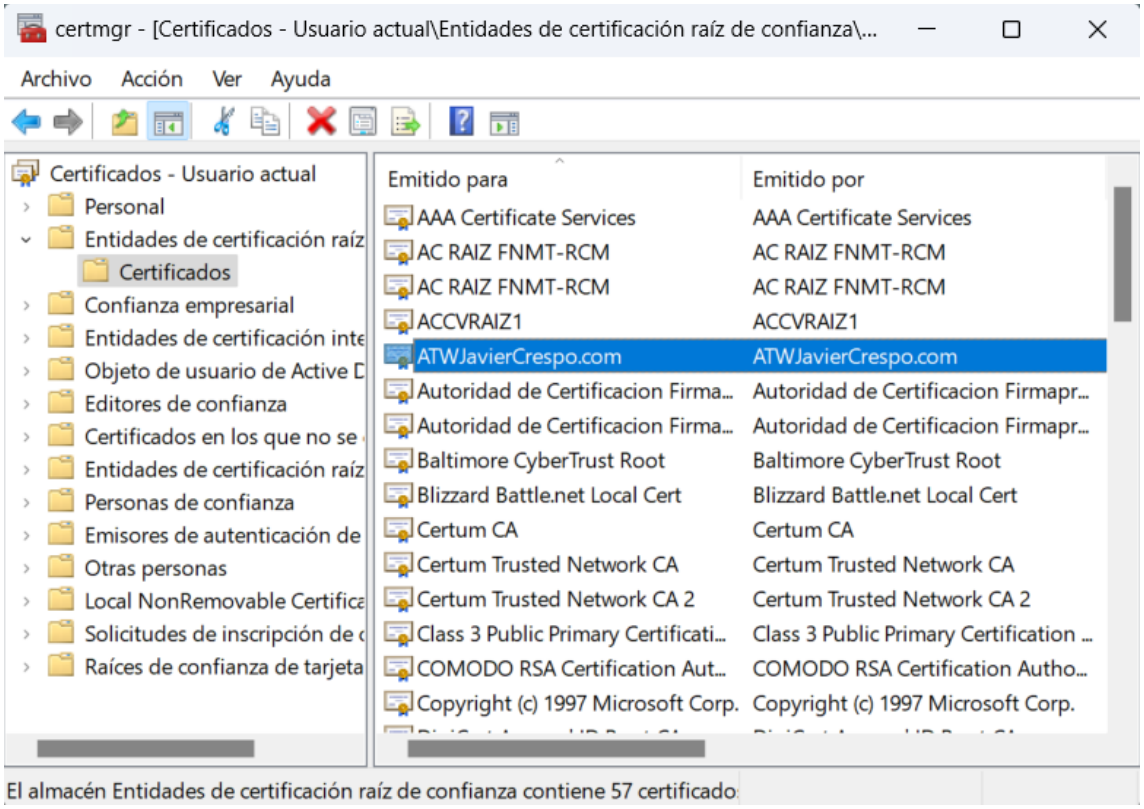

Ilustración 12

Luego exportamos el certificado al directorio bin de Java (donde se encuentra keytool) y lo guardamos con el nombre publica.cer.

Utilizamos el siguiente comando para importar el certificado al almacén de claves de Tomcat:

keytool -import -trustcacerts -alias tomcat -file publica.cer -keystore archivo_keystore

Este comando presentó problemas porque, en lugar de usar ***archivo_keystore***, es necesario especificar la ruta completa del archivo .keystore que creamos previamente.

```
PS C:\Program Files\Java\jdk-23\bin> keytool -import -trustcacerts -alias tomcat -file publica.cer -keystore "C:\atw\.ke
ysotrejcrespo"
Enter keystore password:
Certificate reply was installed in keystore
PS C:\Program Files\Java\jdk-23\bin>
```

Ilustración 13

Este comando añadirá el certificado al almacén de claves especificado, lo que permitirá que Tomcat lo utilice para establecer conexiones seguras. Después de ejecutar este comando, se te pedirá confirmar si deseas confiar en el certificado, a lo cual deberás responder "sí". Una vez hecho esto, el certificado estará correctamente importado.

Archivo hosts

Finalmente, accedemos al archivo **hosts**, ubicado en la ruta *C:\Windows\System32\drivers\etc*, y lo editamos utilizando el Bloc de notas en modo administrador. En este archivo añadimos la siguiente línea: ***127.0.0.1 ATWJavierCrespo.com***

Esta línea vincula el dominio ATWJavierCrespo.com (el nombre del certificado que configuramos) con la dirección local 127.0.0.1, permitiendo que al escribir ese dominio en el navegador, se redirija a nuestro servidor local.

```
Archivo    Editar    Ver

# Copyright (c) 1993-2009 Microsoft Corp.
#
# This is a sample HOSTS file used by Microsoft TCP/IP for Windows.
#
# This file contains the mappings of IP addresses to host names. Each
# entry should be kept on an individual line. The IP address should
# be placed in the first column followed by the corresponding host name.
# The IP address and the host name should be separated by at least one
# space.
#
# Additionally, comments (such as these) may be inserted on individual
# lines or following the machine name denoted by a '#' symbol.
#
# For example:
#
#      102.54.94.97     rhino.acme.com          # source server
#       38.25.63.10     x.acme.com              # x client host

# localhost name resolution is handled within DNS itself.
        127.0.0.1       ATWJavierCrespo.com
#       ::1             localhost
```

Ilustración 14

Problema encontrado

A pesar de realizar esta modificación correctamente, surgió un error al intentar acceder a ATWJavierCrespo.com desde el navegador, ya que no se mostraba la página de bienvenida de Tomcat. El problema radica en que al momento de crear el archivo .keystore, utilicé ATWJavierCrespo.com con mayúsculas, por lo que el nombre de mi certificado se llamara igual como podemos ver en la *ilustración 12*. Sin embargo, el navegador convierte automáticamente las direcciones URL a minúsculas (atwjaviercrespo.com), lo que provoca que no coincidan con el nombre especificado en el certificado.

Solución

Para evitar este problema, debí haber ingresado el dominio completamente en minúsculas (atwjaviercrespo.com) al crear el archivo .keystore. Esto habría asegurado la coincidencia entre el nombre del certificado y la dirección utilizada en el navegador, permitiendo que el servidor Tomcat cargara correctamente la página de bienvenida.

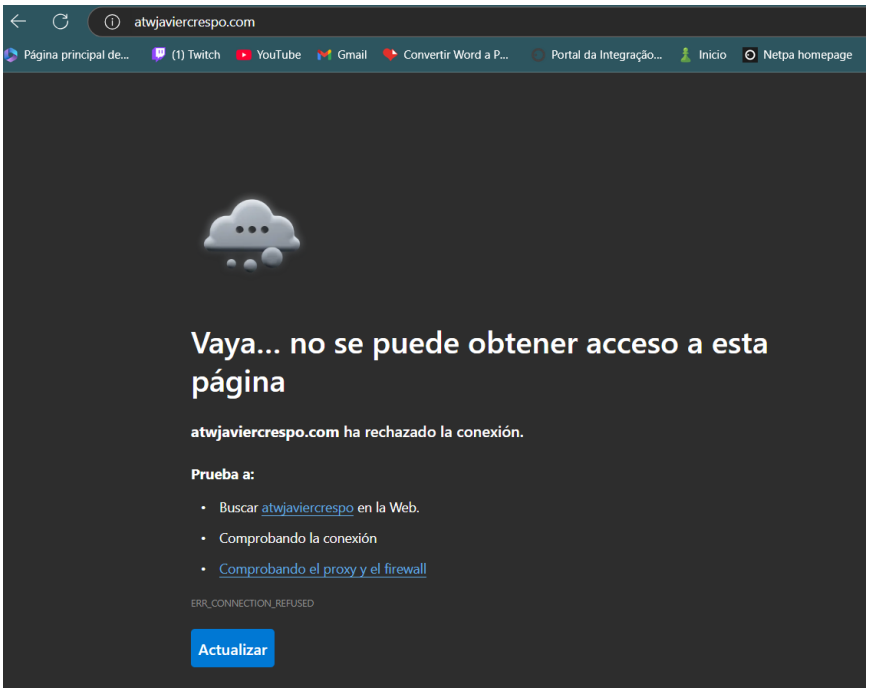

Ilustración 15